# Introduction to Machine Learning
# Unit 7 Problems: Gradient Calculations and Nonlinear Optimization

Shan Qureshi

September 2026

1. (a)

$$\frac{\partial J}{\partial w_i} = \frac{\partial J}{\partial z_1}\frac{\partial z_1}{\partial w_i} + \frac{\partial J}{\partial z_2}\frac{\partial z_2}{\partial w_i}$$
$$\frac{\partial J}{\partial z_1} = (1 + z_1 z_2)e^{z_1 z_2}$$
$$\frac{\partial J}{\partial z_2} = z_1^2 e^{z_1 z_2}$$
$$\frac{\partial z_1}{\partial w_1} = a_1 w_2, \; \frac{\partial z_1}{\partial w_2} = a_1 w_1$$
$$\frac{\partial z_2}{\partial w_1} = a_2, \; \frac{\partial z_2}{\partial w_2} = 2a_3 w_2$$

The final answers are:

$$\frac{\partial J}{\partial w_1} = e^{z_1 z_2}(1 + z_1 z_2)(a_1 w_2) + (z_1^2 e^{z_1 z_2})(a_2)$$
$$\frac{\partial J}{\partial w_2} = e^{z_1 z_2}(1 + z_1 z_2)(a_1 w_1) + (z_1^2 e^{z_1 z_2})(2a_3 w_2)$$

(b)

```python
import numpy as np

def Jeval(w, a):
    w1, w2 = w[0], w[1]
    a1, a2, a3 = a[0], a[1], a[2]

    z1 = a1 * w1 * w2
    z2 = a2 * w1 + a3 * (w2 ** 2)

    J = z1 * np.exp(z1 * z2)

    dJ_dz1 = np.exp(z1 * z2) * (1 + z1 * z2)
```

```
    dJ_dz2 = (z1 ** 2) * np.exp(z1 * z2)

    dz1_dw1 = a1 * w2
    dz1_dw2 = a1 * w1
    dz2_dw1 = a2
    dz2_dw2 = 2 * a3 * w2

    dJ_dw1 = dJ_dz1 * dz1_dw1 + dJ_dz2 * dz2_dw1
    dJ_dw2 = dJ_dz1 * dz1_dw2 + dJ_dz2 * dz2_dw2

    return J, np.array([dJ_dw1, dJ_dw2])
```

2. (a)

$$\frac{\partial J}{\partial \hat{y}_i} = -2(\log(y_i) - \log(\hat{y}_i)) \left(\frac{1}{\hat{y}_i}\right)$$

and we have:

$$\frac{\partial \hat{y}_i}{\partial w_j} = x_{ij}$$
$$\frac{\partial \hat{y}_i}{\partial b} = 1$$

The final gradients are:

$$\frac{\partial J}{\partial w_j} = -2 \sum_{i=1}^{N} \frac{(\log(y_i) - \log(\hat{y}_i))}{\hat{y}_i} x_{ij}$$
$$\frac{\partial J}{\partial b} = -2 \sum_{i=1}^{N} \frac{(\log(y_i) - \log(\hat{y}_i))}{\hat{y}_i}$$

(b)

```
import numpy as np

def Jeval(w, b, x, y):
    y_hat = x.dot(w) + b
    error = np.log(y) - np.log(y_hat)
    J = np.sum(error ** 2)

    dJ_dyhat = -2 * error / y_hat

    Jgradw = x.T.dot(dJ_dyhat)
    Jgradb = np.sum(dJ_dyhat)

    return J, Jgradw, Jgradb
```

3. (a)

$$\frac{\partial J}{\partial z_i} = 2 \left(y_i - \frac{1}{z_i}\right) \left(\frac{1}{z_i^2}\right)$$

and the inner derivatives are as follows:

$$\frac{\partial z_i}{\partial w_0} = 1$$
$$\frac{\partial z_i}{\partial w_j} = x_{ij}$$

The final gradients are:

$$\frac{\partial J}{\partial w_0} = \sum_{i=1}^{n} 2\left(y_i - \frac{1}{z_i}\right)\frac{1}{z_i^2}$$
$$\frac{\partial J}{\partial w_j} = \sum_{i=1}^{n} 2\left(y_i - \frac{1}{z_i}\right)\frac{1}{z_i^2}x_{ij}$$

(b)

```python
import numpy as np

def Jeval(w, x, y):
    w0 = w[0]
    w_rest = w[1:]

    z = w0 + x.dot(w_rest)
    error = y - 1 / z
    J = np.sum(error ** 2)

    dJ_dz = 2 * error / (z ** 2)

    Jgrad0 = np.sum(dJ_dz)
    Jgrad1 = x.T.dot(dJ_dz)

    Jgrad = np.hstack((Jgrad0, Jgrad1))
    return J, Jgrad
```

4. (a)

$$\frac{\partial J}{\partial z_i} = \frac{e^{z_i}}{1+e^{z_i}} - y_i$$
$$\frac{\partial z_i}{\partial a_j} = e^{-(x_i-b_j)^2/2}$$
$$\frac{\partial z_i}{\partial b_j} = a_j(x_i - b_j)e^{-(x_i-b_j)^2/2}$$

Then our gradients become:

$$\frac{\partial J}{\partial a_j} = \sum_{i=1}^{N}\left(\frac{e^{z_i}}{1+e^{z_i}} - y_i\right)e^{-(x_i-b_j)^2/2}$$
$$\frac{\partial J}{\partial b_j} = \sum_{i=1}^{N}\left(\frac{e^{z_i}}{1+e^{z_i}} - y_i\right)a_j(x_i - b_j)e^{-(x_i-b_j)^2/2}$$

(b)

```python
import numpy as np

def Jeval(a, b, x, y):
    diff = x[:, None] - b[None, :]
    K = np.exp(-0.5 * (diff ** 2))
    z = K.dot(a)

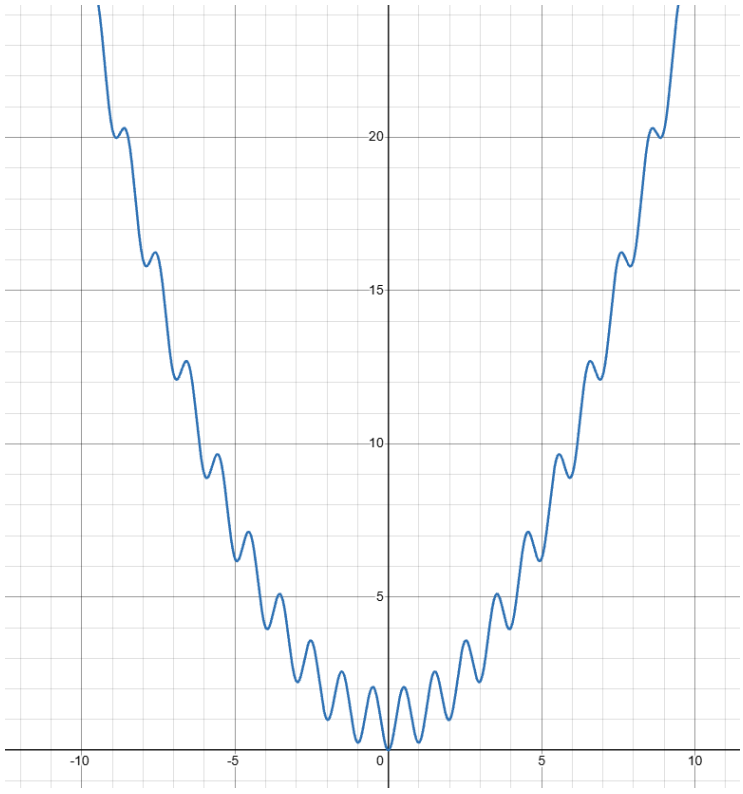
    J = np.sum(np.log(1 + np.exp(z)) - y * z)

    dJ_dz = (1 / (1 + np.exp(-z))) - y

    Jgrada = dJ_dz.dot(K)
    Jgradb = dJ_dz.dot(K * diff * a[None, :])

    return J, Jgrada, Jgradb
```

5. (a)




(b)

$$x_{k+1} = x_k - \alpha \left( \frac{1}{2} x_k + 2\pi \sin(2\pi x_k) \right)$$

(c) Global minima is at $x = 0$

(d) Any parabolic valley near $x = 0$ would pose a problem.

6. (a)

$$\nabla J(w) = \begin{bmatrix} b_1 w_1 \\ b_2 w_2 \end{bmatrix}$$

(b)

$$w^* = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

(c)

$$w_1^{k+1} = (1 - \alpha b_1) w_1^k$$
$$w_2^{k+1} = (1 - \alpha b_2) w_2^k$$

with $\rho_i = 1 - \alpha b_i$

(d) $0 < \alpha < 2/b_2$

(e)

$$\rho_1 = \frac{b_2 - b_1}{b_1 + b_2}$$
$$\rho_2 = -\frac{b_2 - b_1}{b_1 + b_2}$$

7. (a)

$$\nabla_P z_i = x_i x_i^T$$

(b)

$$\nabla_P J(P) = \sum_{i=1}^{n} \left( \frac{1}{y_i} - \frac{1}{z_i} \right) x_i x_i^T$$

(c)

```
import numpy as np

def Jeval_loop(P, x, y):
    n, m = x.shape
    J = 0
    gradP = np.zeros((m, m))

    for i in range(n):
        xi = x[i]
        yi = y[i]
        zi = xi.T.dot(P).dot(xi)

        J += (zi / yi) - np.log(zi)
        gradP += (1 / yi - 1 / zi) * np.outer(xi, xi
            )

    return J, gradP
```

```python
import numpy as np

def Jeval_vec(P, x, y):
    z = np.sum(x.dot(P.T) * x, axis=1)

    J = np.sum((z / y) - np.log(z))
    weights = (1 / y) - (1 / z)

    gradP = (x.T * weights).dot(x)

    return J, gradP
```

(8) (a)

$$\nabla_{w_1} J_1(w_1) = \nabla_{w_1} J(w_1, \hat{w}_2) + \nabla_{w_2} J(w_1, \hat{w}_2) \nabla_{w_1} \hat{w}_2(w_1)$$

However, since $\nabla_{w_2} J(w_1, \hat{w}_2) = 0$, this results in:

$$\nabla_{w_1} J_1(w_1) = \nabla_{w_1} J(w_1, \hat{w}_2)$$